

Greedy Solutions By Pattern

Solutions Grouped by Pattern

Total Problems: 29

Constructive Greedy (3)

CanPlaceFlowers

```
package leetcode.greedy.constructive;
```

```
public class CanPlaceFlowers {
```

```
    /**
     * LeetCode Problem 605: Can Place Flowers
     * <p>
     * You have a long flowerbed in which some of the plots are planted, and some
     * are not. However, flowers cannot be planted in adjacent plots.
     * <p>
     * Given an integer array flowerbed containing 0s and 1s, where 0 means empty
     * and 1 means not empty, and an integer n, return if n new flowers can be
     * planted in the flowerbed without violating the no-adjacent-flowers rule.
     * <p>
     * Greedy Choice Property: iterate through the flowerbed and plant a flower in the first available spot you find.
     * <p>
     * Example 1:
     * Input: flowerbed = [1,0,0,0,1], n = 1
     * Output: true
     * <p>
     * Example 2:
     * Input: flowerbed = [1,0,0,0,1], n = 2
     * Output: false
     */
    public static boolean canPlaceFlowers(int[] flowerbed, int n) {
        // We must loop through the entire flowerbed.
        for (int i = 0; i < flowerbed.length; i++) {
            // Only consider empty plots.
            if (flowerbed[i] == 0) {
                // Check if the left plot is empty (or it's the edge).
                boolean prevEmpty = (i == 0) || (flowerbed[i - 1] == 0);
                // Check if the right plot is empty (or it's the edge).
                boolean nextEmpty = (i == flowerbed.length - 1) || (flowerbed[i + 1] == 0);

                // If both sides are clear, plant a flower.
                if (prevEmpty && nextEmpty) {
                    flowerbed[i] = 1; // Mark the spot as filled.
                    n--; // Decrement the count of flowers to plant.
                }
            }
            // If we've planted all the flowers, we can stop early.
            if (n <= 0) {
                return true;
            }
        }

        // If we finish the loop and n is 0 or less, we succeeded.
        return n <= 0;
    }

    static void main(String[] args) {
        int[] flowerbed = {1, 0, 0, 0, 0, 1};
        int n = 2;

        System.out.println(canPlaceFlowers(flowerbed, n));
    }
}
```

GroupPeople

```
package leetcode.greedy.constructive;
```

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
```

```
/**
 * 1282. Group the People Given the Group Size They Belong To
 * https://leetcode.com/problems/group-the-people-given-the-group-size-they-belong-to/

```

```

*
* There are n people that are split into some unknown number of groups.
* Each person is labeled with a unique ID from 0 to n-1.
*
* You are given an integer array groupSizes, where groupSizes[i] is the size
* of the group that person i is in.
*
* Return a list of groups such that each person i is in a group of size groupSizes[i].
* Each person should appear in exactly one group, and every person must be in a group.
* If there are multiple answers, return any of them.
*
* Example:
*   Input: groupSizes = [3,3,3,3,3,1,3]
*   Output: [[5],[0,1,2],[3,4,6]]
*
* Constraints:
*   groupSizes.length == n
*   1 <= n <= 500
*   1 <= groupSizes[i] <= n
*
* ---
* Approach: Greedy grouping by size
*
* Use a map: groupSize -> list of person IDs.
* When a list reaches its target size, flush it to the result and start a new list.
*
* Greedy choice: assign each person to the first available group of their size.
*
* Time: O(n)
* Space: O(n)
*/
public class GroupPeople {

    public List<List<Integer>> groupThePeople(int[] groupSizes) {
        List<List<Integer>> result = new ArrayList<>();
        Map<Integer, List<Integer>> map = new HashMap<>();

        for (int i = 0; i < groupSizes.length; i++) {
            List<Integer> tempList = map.computeIfAbsent(groupSizes[i], (key) -> new ArrayList<>());
            tempList.add(i);

            if (tempList.size() == groupSizes[i]) {
                result.add(map.get(groupSizes[i]));
                map.put(groupSizes[i], new ArrayList<>());
            }
        }

        return result;
    }
}

```

WiggleSubsequence

```
package leetcode.greedy.constructive;
```

```

/**
 * LeetCode Problem: 376. Wiggle Subsequence
 * <p>
 * Question:
 * A wiggle sequence is a sequence where the differences between successive numbers strictly alternate between positive and negative.
 * The first difference (if one exists) may be either positive or negative. A sequence with one or two differing elements is a wiggle sequence.
 * Given an integer array `nums`, return the length of the longest wiggle subsequence of `nums`.
 * <p>
 * Example:
 * Input: nums = [1, 7, 4, 9, 2, 5]
 * Output: 6
 * Explanation: The entire sequence is a wiggle sequence with differences (6, -3, 5, -7, 3).
 * <p>
 * Constraints:
 * 1 <= nums.length <= 1000
 * 0 <= nums[i] <= 1000
 */
public class WiggleSubsequence {

    public static int wiggleMaxLength(int[] nums) {
        // --- The Greedy Strategy ---
        // Instead of building the actual subsequence, we can just count its length.
        // A wiggle subsequence is made of alternating "peaks" (upward slopes) and "valleys" (downward slopes).
    }
}

```

```

// We can track the length of the longest wiggle sequence ending in a peak and the longest ending in a valley.
// Any single number is a wiggle sequence of length 1, so we start both counts at 1.

if (nums.length < 2) {
    return nums.length;
}

// `peak` stores the length of the longest wiggle subsequence ending with an upward trend (nums[i] > nums[i-1]).
int peak = 1;
// `valley` stores the length of the longest wiggle subsequence ending with a downward trend (nums[i] < nums[i-1]).
int valley = 1;

// Iterate through the array starting from the second element.
for (int i = 1; i < nums.length; ++i) {
    // Case 1: Current element is a PEAK (forms an upward slope).
    if (nums[i] > nums[i - 1]) {
        // To form a new peak, we must extend a previous valley.
        // So, the new peak's length is the length of the best valley sequence so far, plus one (for the current element).
        peak = valley + 1;
    }
    // Case 2: Current element is a VALLEY (forms a downward slope).
    else if (nums[i] < nums[i - 1]) {
        // To form a new valley, we must extend a previous peak.
        // So, the new valley's length is the length of the best peak sequence so far, plus one.
        valley = peak + 1;
    }
    // Case 3: nums[i] == nums[i-1] (a flat slope).
    // A flat point cannot extend a wiggle sequence, so we do nothing and the `peak` and `valley` lengths remain unchanged.
}

// The longest wiggle subsequence could end in either a peak or a valley.
// We return the maximum of the two possibilities.
return Math.max(peak, valley);
}

static void main(String[] args) {
    int[] nums = {1, 7, 4, 9, 2, 5};
    System.out.println("Longest wiggle subsequence for [1, 7, 4, 9, 2, 5]: " + wiggleMaxLength(nums)); // Expected: 6

    int[] nums2 = {1, 17, 5, 10, 13, 15, 10, 5, 16, 8};
    System.out.println("Longest wiggle subsequence for [1, 17, 5, ...]: " + wiggleMaxLength(nums2)); // Expected: 7
}
}

```

Specialized Greedy (5)

CanJump

```
package leetcode.greedy.specialized;

/**
 * 55. Jump Game
 * https://leetcode.com/problems/jump-game/
 *
 * You are given an integer array nums. You are initially positioned at the
 * array's first index, and each element represents your maximum jump length
 * at that position. Return true if you can reach the last index.
 *
 * Example 1: nums = [2,3,1,1,4] -> true (jump 1 to index 1, then 3 to last)
 * Example 2: nums = [3,2,1,0,4] -> false (stuck at index 3)
 *
 * Constraints:
 * 1 <= nums.length <= 10^4
 * 0 <= nums[i] <= 10^5
 *
 * ---
 * Approach: Greedy -- track farthest reachable index
 *
 * Maintain 'far' = the farthest index reachable so far.
 * At each position i:
 * - If i > far, we can't reach this position -> return false.
 * - Update far = max(far, i + nums[i]).
 *
 * Time: O(n)
 * Space: O(1)
 */
public class CanJump {

    public static boolean canJump(int[] nums) {
        int far = 0;
        int n = nums.length;

        for (int i = 0; i < n; i++) {
            if (i > far)
                return false;
            far = Math.max(i + nums[i], far);
        }

        return true;
    }

    public static void main(String[] args) {
        int[] nums = { 3, 2, 1, 1, 4 };
        System.out.println(canJump(nums));
    }
}
```

CanJump_2

```
package leetcode.greedy.specialized;

/**
 * 45. Jump Game II
 * https://leetcode.com/problems/jump-game-ii/
 *
 * You are given a 0-indexed array of integers nums of length n. You are
 * initially positioned at nums[0]. Each element represents the max length
 * of a forward jump from that position.
 *
 * Return the minimum number of jumps to reach nums[n-1].
 * The test cases are generated such that you can always reach the last index.
 *
 * Example 1: nums = [2,3,1,1,4] -> 2 (jump 1 to index 1, then 3 to index 4)
 * Example 2: nums = [2,3,0,1,4] -> 2
 *
 * Constraints:
 * 1 <= nums.length <= 10^4
 * 0 <= nums[i] <= 1000
 */
```

```

*
* ---
* Approach: Greedy -- BFS-like level expansion
*
* Think of it as BFS where each "level" is one jump:
* - 'reached' = the farthest index reachable with current number of jumps.
* - 'far' = the farthest index reachable with one more jump.
* - When i == reached, we must take a jump: count++ and reached = far.
*
* Time: O(n)
* Space: O(1)
*/
public class CanJump_2 {

    public static int canJump(int[] nums) {
        int far = 0;
        int reached = 0;
        int count = 0;
        int n = nums.length;

        for (int i = 0; i < n - 1; i++) {
            far = Math.max(i + nums[i], far);

            if (i == reached) {
                count++;
                reached = far;
            }
        }

        return count;
    }

    public static void main(String[] args) {
        int[] nums = { 2, 3, 1, 1, 4 };
        System.out.println(canJump(nums));
    }
}

```

Candy

```

package leetcode.greedy.specialized;

/**
 * 135. Candy
 * https://leetcode.com/problems/candy/
 *
 * There are n children standing in a line. Each child is assigned a rating value
 * given in the integer array ratings.
 *
 * You are giving candies to these children subjected to the following requirements:
 * 1. Each child must have at least one candy.
 * 2. Children with a higher rating than their neighbors get more candies.
 *
 * Return the minimum number of candies you need to have to distribute.
 *
 * Example 1: ratings = [1,0,2] -> 5 (candies = [2,1,2])
 * Example 2: ratings = [1,2,2] -> 4 (candies = [1,2,1])
 *
 * Constraints:
 * n == ratings.length
 * 1 <= n <= 2 * 10^4
 * 0 <= ratings[i] <= 2 * 10^4
 *
 * ---
 * Approach: Two-pass greedy
 *
 * Pass 1 (left to right): If ratings[i] > ratings[i-1], give one more candy than left neighbor.
 * Pass 2 (right to left): If ratings[i] > ratings[i+1], ensure at least one more than right neighbor.
 *
 * Each pass satisfies one direction of the constraint. Together they satisfy both.
 *
 * Time: O(n)
 * Space: O(n)
 */
public class Candy {

    public static int candy(int[] ratings) {
        int[] candies = new int[ratings.length];
        int totalCandy = 0;
    }
}

```

```

candies[0] = 1;

// Traverse from left to right, check if left neighbour is greater than current element, If yes then add prev candies plus 1
for (int i = 1; i < ratings.length; i++) {
    if (ratings[i] > ratings[i - 1]) {
        candies[i] = candies[i - 1] + 1;
    } else {
        candies[i] = 1;
    }
}

// Traverse from right to left, check if right neighbour is greater than current element, If yes then add max of current candy
for (int i = ratings.length - 2; i >= 0; i--) {
    if (ratings[i] > ratings[i + 1]) {
        candies[i] = Math.max(candies[i], candies[i + 1] + 1);
    }
}

for (int candy : candies) {
    totalCandy += candy;
}

return totalCandy;

}

public static void main(String[] args) {
    int[] ratings = {1, 3, 4, 5, 2};
    System.out.println(candy(ratings));
}
}

```

GasStation

```

package leetcode.greedy.specialized;

/**
 * 134. Gas Station
 * https://leetcode.com/problems/gas-station/
 *
 * There are n gas stations along a circular route, where the amount of gas at
 * the i-th station is gas[i]. You have a car with an unlimited gas tank and it
 * costs cost[i] of gas to travel from the i-th station to (i+1)-th station.
 *
 * You begin the journey with an empty tank at one of the gas stations.
 * Given two integer arrays gas and cost, return the starting gas station's index
 * if you can travel around the circuit once in the clockwise direction,
 * otherwise return -1. If a solution exists, it is guaranteed to be unique.
 *
 * Example 1:
 * Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
 * Output: 3
 * Explanation: Start at station 3 (gas=4).
 * Station 3: tank = 0+4-1 = 3
 * Station 4: tank = 3+5-2 = 6
 * Station 0: tank = 6+1-3 = 4
 * Station 1: tank = 4+2-4 = 2
 * Station 2: tank = 2+3-5 = 0 (made it!)
 *
 * Example 2:
 * Input: gas = [2,3,4], cost = [3,4,3]
 * Output: -1 (total gas < total cost)
 *
 * Constraints:
 * n == gas.length == cost.length
 * 1 <= n <= 10^5
 * 0 <= gas[i], cost[i] <= 10^4
 *
 * ---
 * Approach: Greedy -- single pass
 *
 * Key observations:
 * 1. If total gas < total cost, no solution exists.
 * 2. If we start at station s and can't reach station t (tank goes negative),
 * then NO station between s and t can be the answer either.
 * -> Jump start to t+1.
 *
 */

```

```

* Algorithm:
*   - Track totalSurplus (gas - cost for all stations) to check feasibility.
*   - Track currentTank. If it goes negative, reset start to next station.
*
* Time: O(n)
* Space: O(1)
*/
public class GasStation {

    public static int canCompleteCircuit(int[] gas, int[] cost) {
        int totalSurplus = 0; // Check if solution is possible at all
        int currentTank = 0; // Track tank for current candidate start
        int start = 0; // Candidate starting station

        for (int i = 0; i < gas.length; i++) {
            int surplus = gas[i] - cost[i];
            totalSurplus += surplus;
            currentTank += surplus;

            if (currentTank < 0) {
                // Can't reach station i+1 from current start.
                // No station between start and i works either.
                // Try starting from i+1.
                start = i + 1;
                currentTank = 0;
            }
        }

        // If total gas >= total cost, the answer is guaranteed to be 'start'
        return totalSurplus >= 0 ? start : -1;
    }

    public static void main(String[] args) {
        int[] gas1 = {1, 2, 3, 4, 5};
        int[] cost1 = {3, 4, 5, 1, 2};
        System.out.println("Example 1: " + canCompleteCircuit(gas1, cost1)); // 3

        int[] gas2 = {2, 3, 4};
        int[] cost2 = {3, 4, 3};
        System.out.println("Example 2: " + canCompleteCircuit(gas2, cost2)); // -1
    }
}

```

QueueReconstruction

```

package leetcode.greedy.specialized;

import java.util.Arrays;
import java.util.LinkedList;
import java.util.List;

/**
 * 406. Queue Reconstruction by Height
 * https://leetcode.com/problems/queue-reconstruction-by-height/
 *
 * You are given an array of people, people, which are the attributes of some
 * people in a queue (not necessarily in order). Each people[i] = [hi, ki]
 * represents the i-th person of height hi with exactly ki other people in front
 * who have a height greater than or equal to hi.
 *
 * Reconstruct and return the queue.
 *
 * Example:
 * Input: people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
 * Output: [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
 *
 * Constraints:
 * 1 <= people.length <= 2000
 * 0 <= hi <= 10^6
 * 0 <= ki < people.length
 *
 * ---
 * Approach: Sort descending by height, then insert by k-value
 *
 * 1. Sort by height descending (tallest first). Break ties by k ascending.
 * 2. Insert each person at index k in the result list.
 *
 * Why this works: When we insert a shorter person, all previously-inserted
 * people are taller or equal, so inserting at index k guarantees exactly k
 * taller people before them.

```

```

*
* Time:  $O(n^2)$  due to list insertions
* Space:  $O(n)$ 
*/
public class QueueReconstruction {

    public static int[][] reconstructQueue(int[][] people) {
        Arrays.sort(people, (a, b) -> {
            if (a[0] != b[0]) return b[0] - a[0]; // Sort by height descending
            return a[1] - b[1]; // Sort by k ascending
        });

        List<int[]> list = new LinkedList<>();
        for (int[] current : people) {
            list.add(current[1], new int[]{current[0], current[1]});
        }

        return list.toArray(new int[people.length][]);
    }

    public static void main(String[] args) {
        int[][] people = {{7, 0}, {4, 4}, {7, 1}, {5, 0}, {6, 1}, {5, 2}};
        System.out.println(Arrays.deepToString(reconstructQueue(people)));
    }
}

```

Stack-Based Greedy (2)

RemoveDuplicateLetters

```
package leetcode.greedy.stack;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Stack;

/**
 * 316. Remove Duplicate Letters
 * https://leetcode.com/problems/remove-duplicate-letters/
 *
 * Given a string s, remove duplicate letters so that every letter appears once
 * and only once. You must make sure your result is the smallest in
 * lexicographical order among all possible results.
 *
 * Example 1: s = "bcabc" -> "abc"
 * Example 2: s = "cbacdcbc" -> "acdb"
 *
 * Constraints:
 * 1 <= s.length <= 10^4
 * s consists of lowercase English letters.
 *
 * ---
 * Approach: Monotonic stack + greedy
 *
 * Use a stack to build the result. For each character:
 * 1. If already in stack, skip (duplicates handled).
 * 2. While stack top > current char AND stack top appears later in string,
 *    pop it (we can add it back later for a smaller result).
 * 3. Push current char.
 *
 * "Appears later" is tracked via a remaining-count map.
 *
 * This is the SAME problem as LC 1081 (Smallest Subsequence of Distinct Characters).
 *
 * Time: O(n)
 * Space: O(1) (26 letters max in stack)
 */
public class RemoveDuplicateLetters {

    public static String removeDuplicateLetters(String s) {
        HashMap<Character, Integer> map = new HashMap<>();
        for (char c : s.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        Stack<Character> stack = new Stack<>();
        HashSet<Character> seen = new HashSet<>();

        for (char c : s.toCharArray()) {
            map.put(c, map.get(c) - 1);
            if (seen.contains(c)) continue;

            while (!stack.isEmpty() && stack.peek() > c && map.get(stack.peek()) > 0) {
                seen.remove(stack.pop());
            }

            stack.push(c);
            seen.add(c);
        }

        StringBuilder result = new StringBuilder();
        for (Character c : stack) {
            result.append(c);
        }

        return result.toString();
    }

    public static void main(String[] args) {
        String s = "cbacdcbc";
        System.out.println(removeDuplicateLetters(s));
    }
}
```

```
}
```

SmallestSubsequenceDistinctCharacters

```
package leetcode.greedy.stack;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;
import java.util.Stack;

/**
 * 1081. Smallest Subsequence of Distinct Characters
 * https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/
 *
 * Return the lexicographically smallest subsequence of s that contains all
 * the distinct characters of s exactly once.
 *
 * This is identical to LC 316 (Remove Duplicate Letters).
 *
 * Example 1: s = "bcabc" -> "abc"
 * Example 2: s = "cbacdcbc" -> "acdb"
 *
 * Constraints:
 * 1 <= s.length <= 1000
 * s consists of lowercase English letters.
 *
 * ---
 * Approach: Monotonic stack + greedy (same as LC 316)
 *
 * Track last index of each char. Use a stack:
 * - Skip chars already in stack.
 * - Pop stack top if it's > current char AND it appears later.
 * - Push current char.
 *
 * Time: O(n)
 * Space: O(1)
 */
public class SmallestSubsequenceDistinctCharacters {

    public static String smallestSubsequence(String s) {
        Stack<Character> stack = new Stack<>();
        Set<Character> seen = new HashSet<>();
        Map<Character, Integer> lastIndex = new HashMap<>();

        // Build the last index map
        for (int i = 0; i < s.length(); i++) {
            lastIndex.put(s.charAt(i), i);
        }

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (seen.contains(c))
                continue;

            while (!stack.isEmpty() && c < stack.peek() && lastIndex.get(stack.peek()) > i) {
                seen.remove(stack.pop());
            }

            stack.push(c);
            seen.add(c);
        }

        // Build result
        StringBuilder result = new StringBuilder();
        for (char c : stack)
            result.append(c);

        return result.toString();
    }

    public static void main(String[] args) {
        String s = "cbacdcbc";
        System.out.println(smallestSubsequence(s));
    }
}
```

Frequency Counting (6)

ArrayDoubledPairs

```
package leetcode.greedy.frequencyCounting;

import java.util.Arrays;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Map;

/**
 * 954. Array of Doubled Pairs
 * https://leetcode.com/problems/array-of-doubled-pairs/
 *
 * Given an integer array of even length arr, return true if it is possible
 * to reorder arr such that arr[2*i + 1] = 2 * arr[2*i] for every 0 <= i < len(arr) / 2.
 *
 * In other words, can we pair every element x with 2x?
 *
 * Example 1: arr = [3,1,3,6] -> false
 * Example 2: arr = [2,1,2,6] -> false
 * Example 3: arr = [4,-2,2,-4] -> true ([-2,-4],[2,4])
 * Example 4: arr = [1,2,4,16,8,4] -> false
 *
 * Constraints:
 * 2 <= arr.length <= 3 * 10^4
 * arr.length is even.
 * -10^5 <= arr[i] <= 10^5
 *
 * ---
 * Approach: Greedy + Sort by absolute value + Frequency map
 *
 * 1. Count frequency of each number.
 * 2. Sort by absolute value (so we process smaller magnitudes first).
 * 3. For each x, try to pair it with 2x. If freq[2x] < freq[x], impossible.
 *
 * Why sort by |x|? For negatives: -4 pairs with -2 (half), not -8 (double).
 * Sorting by |x| ensures we always process x before 2x.
 *
 * Time: O(n log n)
 * Space: O(n)
 */
public class ArrayDoubledPairs {

    public static boolean canReorderDoubled(int[] arr) {
        Map<Integer, Integer> map = new HashMap<>();
        // Step 1: Count frequency of each number
        for (int num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        // Step 2: Sort by absolute value using boxed stream
        Integer[] s = Arrays.stream(arr)
            .boxed()
            .sorted(Comparator.comparingInt(Math::abs))
            .toArray(Integer[]::new);

        // Step 3: Try to pair each number with its double
        for (int x : s) {
            if (map.get(x) == 0)
                continue;

            int doubleX = 2 * x;

            if (map.getOrDefault(doubleX, 0) < map.get(x)) {
                return false;
            }
            map.put(doubleX, map.get(doubleX) - map.get(x));
            map.put(x, 0);
        }

        return true;
    }

    public static void main(String[] args) {
        int[] arr = { 2, 4, 0, 0, 8, 1 };
    }
}
```

```

        // arr = new int[] {1, 2, 4, 16, 8, 4};
        System.out.println(canReorderDoubled(arr));
    }
}

```

ConstructKPalindromeStrings

package leetcode.greedy.frequencyCounting;

import java.util.HashMap;

import java.util.Map;

```

/**
 * 1400. Construct K Palindrome Strings
 * https://leetcode.com/problems/construct-k-palindrome-strings/
 *
 * Given a string s and an integer k, return true if you can use all the
 * characters in s to construct k palindrome strings.
 *
 * Example 1: s = "annabelle", k = 2 -> true ("anna" + "elble")
 * Example 2: s = "leetcode", k = 3 -> false
 * Example 3: s = "true", k = 4 -> true ("t","r","u","e")
 *
 * Constraints:
 * 1 <= s.length <= 10^5
 * 1 <= k <= 10^5
 *
 * ---
 * Approach: Count odd-frequency characters
 *
 * Key insight: A palindrome can have at most 1 character with odd frequency (the center).
 * So to form k palindromes, we need at most k characters with odd frequency.
 *
 * Conditions:
 * - k > s.length() -> impossible (not enough chars)
 * - Count of odd-frequency chars > k -> impossible
 * - Otherwise -> possible
 *
 * Time: O(n)
 * Space: O(1) (26 letters)
 */
public class ConstructKPalindromeStrings {

    public boolean canConstruct(String s, int k) {
        if (s.length() == k) return true;
        if (k > s.length()) return false;

        Map<Character, Integer> map = new HashMap<>();

        for (char c : s.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        int odd = 0;
        for (Map.Entry<Character, Integer> entry : map.entrySet()) {
            if (entry.getValue() % 2 != 0) odd++;

            if (odd > k) return false;
        }

        return true;
    }

    public boolean canConstructOptimized(String s, int k) {
        if (s.length() == k) return true;
        if (k > s.length()) return false;

        int odds = 0;
        int[] counts = new int[26];

        for (int i = 0; i < s.length(); i++) {
            int index = s.charAt(i) - 'a';
            counts[index]++;
            odds += counts[index] % 2 == 0 ? -1 : 1;
        }

        return odds <= k;
    }
}

```

```

    }
}

```

DivideArraySetsKConsecutiveNumbers

```

package leetcode.greedy.frequencyCounting;

import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 1296. Divide Array in Sets of K Consecutive Numbers
 * https://leetcode.com/problems/divide-array-in-sets-of-k-consecutive-numbers/
 *
 * Given an array of integers nums and a positive integer k, check whether it is
 * possible to divide this array into sets of k consecutive numbers.
 *
 * Example 1: nums = [1,2,3,3,4,4,5,6], k = 4 -> true ([1,2,3,4] and [3,4,5,6])
 * Example 2: nums = [3,2,1,2,3,4,3,4,5,9,10,11], k = 3 -> true
 * Example 3: nums = [1,2,3,4], k = 3 -> false
 *
 * Constraints:
 * 1 <= k <= nums.length <= 10^5
 * 1 <= nums[i] <= 10^9
 *
 * ---
 * Approach: Greedy with min-heap
 *
 * Always start a new group from the smallest available number.
 * Use a min-heap to always access the smallest element.
 * For each group, greedily take k consecutive numbers starting from the minimum.
 *
 * (Can also be solved with TreeMap for O(n log n) instead of O(n * k).)
 *
 * Time: O(n * k) with heap removal
 * Space: O(n)
 */
public class DivideArraySetsKConsecutiveNumbers {

    // Can be solved using tree map
    public boolean isPossibleDivide(int[] nums, int k) {
        Queue<Integer> minHeap = new PriorityQueue<>();
        for (int num : nums) {
            minHeap.add(num);
        }

        while (!minHeap.isEmpty()) {
            int start = minHeap.poll(); // Remove first element
            for (int i = 1; i < k; i++) {
                if (minHeap.remove(start + i)) {
                    continue;
                }
                return false;
            }
        }

        return true;
    }
}

```

ReduceArraySizeHalf

```

package leetcode.greedy.frequencyCounting;

import java.util.*;

/**
 * 1338. Reduce Array Size to The Half
 * https://leetcode.com/problems/reduce-array-size-to-the-half/
 *
 * You are given an integer array arr. You can choose a set of integers and
 * remove all the occurrences of these integers in the array.
 *
 * Return the minimum size of the set so that at least half of the integers
 * of the array are removed.
 */

```

```

* Example 1: arr = [3,3,3,3,5,5,2,2,7] -> 2 (remove {3,7} -> 6 removed >= 5)
* Example 2: arr = [7,7,7,7,7,7] -> 1
*
* Constraints:
*   2 <= arr.length <= 10^5
*   arr.length is even.
*   1 <= arr[i] <= 10^5
*
* ---
* Approach: Greedy -- remove most frequent first
*
* 1. Count frequency of each number.
* 2. Sort frequencies in descending order.
* 3. Greedily remove the most frequent numbers until we've removed >= half.
*
* Time: O(n log n)
* Space: O(n)
*/
public class ReduceArraySizeHalf {

    // The goal is to minimize the number of unique numbers removed.
    public int minSetSize(int[] arr) {
        // Step 1: Count frequency of each number
        Map<Integer, Integer> map = new HashMap<>();
        for (int num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        // Step 2: Store frequencies in a list and sort in descending order
        List<Integer> freqList = new ArrayList<>(map.values());
        freqList.sort(Collections.reverseOrder());

        // Step 3: Remove elements greedily from the highest frequency
        int halfSize = arr.length / 2;
        int removed = 0, setSize = 0;

        for (int freq : freqList) {
            removed += freq;
            setSize++;
            if (removed >= halfSize) {
                return setSize;
            }
        }

        return setSize;
    }

    public static void main(String[] args) {
        ReduceArraySizeHalf sol = new ReduceArraySizeHalf();

        int[] arr1 = {3,3,3,3,5,5,2,2,7};
        System.out.println(sol.minSetSize(arr1)); // Output: 2

        int[] arr2 = {7,7,7,7,7,7};
        System.out.println(sol.minSetSize(arr2)); // Output: 1

        int[] arr3 = {1,9,9,2,2,2,3,3,3,3,4,4,4,4};
        System.out.println(sol.minSetSize(arr3)); // Output: 2
    }
}

```

ReorganizeString

```
package leetcode.greedy.frequencyCounting;
```

```
import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Queue;
```

```

/**
 * LeetCode Problem: 767. Reorganize String
 * <p>
 * Question:
 * Given a string `s`, rearrange the characters of `s` so that any two adjacent characters are not the same.
 * Return any possible rearrangement of `s` or return "" if it is impossible.
 * <p>
 * Example 1:

```

```

* Input: s = "aab"
* Output: "aba"
* <p>
* Example 2:
* Input: s = "aaab"
* Output: ""
* <p>
* Constraints:
* - 1 <= s.length <= 500
* - s consists of lowercase English letters.
*/
public class ReorganizeString {

    public static String reorganizeString(String s) {
        // --- The Greedy Strategy ---
        // The core idea is to always append the most frequent character available that is different
        // from the last character added. A max heap is the perfect data structure for this,
        // as it always gives us the character with the highest remaining frequency.

        // Step 1: Count the frequency of each character.
        Map<Character, Integer> map = new HashMap<>();
        for (char c : s.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        // Step 2: Create a max heap (PriorityQueue) to store characters.
        // The character with the highest frequency will have the highest priority.
        Queue<Character> queue = new PriorityQueue<>((a, b) -> map.get(b) - map.get(a));
        queue.addAll(map.keySet());

        StringBuilder result = new StringBuilder();

        // Step 3: Greedily build the new string.
        // We pull the two most frequent characters from the heap at each step.
        // This ensures that the most frequent character is always separated by the second most frequent one.
        while (queue.size() >= 2) {
            Character char1 = queue.poll();
            Character char2 = queue.poll();

            // Append them to the result
            result.append(char1);
            result.append(char2);

            // Decrement their counts
            map.put(char1, map.get(char1) - 1);
            map.put(char2, map.get(char2) - 1);

            // If the characters still have remaining counts, add them back to the heap.
            // The heap will automatically re-order them based on their new frequencies.
            if (map.get(char1) > 0) {
                queue.add(char1);
            }
            if (map.get(char2) > 0) {
                queue.add(char2);
            }
        }

        // Step 4: Handle the last remaining character, if any.
        if (!queue.isEmpty()) {
            Character lastChar = queue.poll();
            // If the last remaining character has a frequency greater than 1, it's impossible.
            // This means we have, for example, 'a' left, but the string ends in 'a', so we can't place it.
            if (map.get(lastChar) > 1) {
                return "";
            }
            // Otherwise, it's safe to append the last character.
            result.append(lastChar);
        }

        return result.toString();
    }

    static void main(String[] args) {
        String s1 = "aab";
        System.out.println("Reorganized " + s1 + ": " + reorganizeString(s1)); // Expected: "aba"

        String s2 = "aaab";
        System.out.println("Reorganized " + s2 + ": " + reorganizeString(s2)); // Expected: ""

        String s3 = "aaabc";
        System.out.println("Reorganized " + s3 + ": " + reorganizeString(s3)); // Expected: "abaca"
    }
}

```

```
}
}
```

TaskScheduler

```
package leetcode.greedy.frequencyCounting;

/**
 * 621. Task Scheduler
 * https://leetcode.com/problems/task-scheduler/
 *
 * You are given an array of CPU tasks, each represented by a letter (A to Z),
 * and a cooling interval n. Each cycle or interval allows completion of one task.
 * Tasks can be completed in any order, but there's a constraint: identical tasks
 * must be separated by at least n intervals.
 *
 * Return the minimum number of intervals the CPU will take to finish all tasks.
 *
 * Example 1:
 * Input: tasks = ["A","A","A","B","B","B"], n = 2
 * Output: 8
 * Explanation: A -> B -> idle -> A -> B -> idle -> A -> B
 *
 * Example 2:
 * Input: tasks = ["A","C","A","B","D","B"], n = 1
 * Output: 6
 * Explanation: A -> B -> A -> C -> B -> D (no idle needed)
 *
 * Example 3:
 * Input: tasks = ["A","A","A","B","B","B"], n = 3
 * Output: 10
 * Explanation: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B
 *
 * Constraints:
 * 1 <= tasks.length <= 10^4
 * tasks[i] is an uppercase English letter.
 * 0 <= n <= 100
 *
 * ---
 * Approach: Greedy -- math formula based on max frequency
 *
 * Let maxFreq = frequency of the most common task.
 * Let maxCount = number of tasks that have this max frequency.
 *
 * Visualization for tasks=[A,A,A,B,B,B], n=2:
 * A _ _ | A _ _ | A
 * Slots between A's: (maxFreq - 1) groups of size (n + 1)
 * Plus the last group: just the tasks with maxFreq (maxCount)
 *
 * Formula: result = (maxFreq - 1) * (n + 1) + maxCount
 *
 * But if there are many different tasks, we might not need any idle time.
 * In that case, answer = tasks.length.
 *
 * Final answer: max(tasks.length, (maxFreq - 1) * (n + 1) + maxCount)
 *
 * Time: O(n) -- one pass to count frequencies
 * Space: O(1) -- at most 26 letters
 */
public class TaskScheduler {

    public static int leastInterval(char[] tasks, int n) {
        // Count frequency of each task
        int[] freq = new int[26];
        for (char task : tasks) {
            freq[task - 'A']++;
        }

        // Find the maximum frequency
        int maxFreq = 0;
        for (int f : freq) {
            maxFreq = Math.max(maxFreq, f);
        }

        // Count how many tasks share the maximum frequency
        int maxCount = 0;
        for (int f : freq) {
            if (f == maxFreq) maxCount++;
        }
    }
}
```

```

// Formula: (maxFreq - 1) chunks of size (n+1) + maxCount
// Example: A A A B B B, n=2
//   maxFreq=3, maxCount=2
//   (3-1) * (2+1) + 2 = 6 + 2 = 8
//   Layout: [A B _] [A B _] [A B] -> 8 slots
int formulaResult = (maxFreq - 1) * (n + 1) + maxCount;

// If tasks fill all slots naturally (no idle needed), answer = tasks.length
return Math.max(tasks.length, formulaResult);
}

public static void main(String[] args) {
    char[] tasks1 = {'A', 'A', 'A', 'B', 'B', 'B'};
    System.out.println("Example 1: " + leastInterval(tasks1, 2)); // 8

    char[] tasks2 = {'A', 'C', 'A', 'B', 'D', 'B'};
    System.out.println("Example 2: " + leastInterval(tasks2, 1)); // 6

    char[] tasks3 = {'A', 'A', 'A', 'B', 'B', 'B'};
    System.out.println("Example 3: " + leastInterval(tasks3, 3)); // 10
}
}

```

Sorting-Based Greedy (3)

AssignCookies

```
package leetcode.greedy.sorting;

import java.util.Arrays;

/**
 * LeetCode Problem: 455. Assign Cookies
 * <p>
 * Question:
 * Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.
 * Each child `i` has a greed factor `g[i]`, which is the minimum size of a cookie that the child will be content with;
 * and each cookie `j` has a size `s[j]`. If `s[j] >= g[i]`, we can assign the cookie `j` to the child `i`, and the child `i` will be content.
 * Your goal is to maximize the number of your content children and output the maximum number.
 * <p>
 * Example 1:
 * Input: g = [1, 2, 3], s = [1, 1]
 * Output: 1
 * Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
 * You can only satisfy the child with greed factor 1. So you should output 1.
 * <p>
 * Constraints:
 * - 1 <= g.length <= 3 * 10^4
 * - 0 <= s.length <= 3 * 10^4
 * - 1 <= g[i], s[j] <= 2^31 - 1
 */
public class AssignCookies {

    /**
     * Finds the maximum number of content children using a greedy approach.
     *
     * @param greed An array representing the greed factor of each child.
     * @param cookie An array representing the size of each cookie.
     * @return The maximum number of content children.
     */
    public static int findContentChildren(int[] greed, int[] cookie) {
        // --- The Greedy Strategy ---
        // To maximize the number of content children, we should be efficient with our cookies.
        // The best approach is to give the smallest cookie that can satisfy a child to the least greedy child.
        // This strategy saves larger cookies for greedier children who need them.
        // To implement this, we sort both arrays first.

        // Sort the greed factors and cookie sizes in ascending order.
        Arrays.sort(greed);
        Arrays.sort(cookie);

        int j = 0; // Pointer for the cookie array.
        int result = 0; // The count of content children.

        // Iterate through each child's greed factor.
        for (int k : greed) {

            // Find the smallest available cookie that can satisfy the current child.
            // We skip over all cookies that are too small.
            while (j < cookie.length && cookie[j] < k) {
                j++;
            }

            // If we found a suitable cookie (i.e., we haven't run out of cookies)
            if (j < cookie.length) {
                result++; // This child is now content.
                j++; // Move to the next cookie, as this one has been used.
            }
        }
        return result;
    }

    static void main(String[] args) {
        // Test case 1:
        int[] greed1 = {10, 9, 8, 7};
        int[] cookie1 = {5, 6, 7, 8};
        // After sorting: greed = [7, 8, 9, 10], cookie = [5, 6, 7, 8]
        // Child with greed 7 gets cookie 7.
        // Child with greed 8 gets cookie 8.
    }
}
```

```

// No cookies left for children with greed 9 and 10.
System.out.println(findContentChildren(greed1, cookie1)); // Expected: 2

// Test case 2:
int[] greed2 = {1, 2};
int[] cookie2 = {1, 2, 3};
// After sorting: greed = [1, 2], cookie = [1, 2, 3]
// Child with greed 1 gets cookie 1.
// Child with greed 2 gets cookie 2.
System.out.println(findContentChildren(greed2, cookie2)); // Expected: 2
}
}

```

BagOfTokens

```

package leetcode.greedy.sorting;

import java.util.Arrays;

public class BagOfTokens {

    public static int bagOfTokensScore(int[] tokens, int power) {
        Arrays.sort(tokens);
        int left = 0;
        int right = tokens.length - 1;
        int score = 0;

        while (left <= right) {
            // Power up
            if (power < tokens[left] && score > 0) {
                score--;
                power += tokens[right];
                right--;
            }

            // Power down
            if (power >= tokens[left]) {
                power -= tokens[left];
                score++;
            }

            left++;
        }

        return score;
    }

    public static void main(String[] args) {
        int[] tokens = { 100, 200, 300, 400 };
        int power = 200;
        System.out.println(bagOfTokensScore(tokens, power));
    }
}

```

BoatsToSavePeople

```

package leetcode.greedy.sorting;

import java.util.Arrays;

/**
 * 881. Boats to Save People
 * https://leetcode.com/problems/boats-to-save-people/
 *
 * You are given an array people where people[i] is the weight of the i-th person,
 * and an infinite number of boats where each boat can carry a maximum weight of limit.
 * Each boat carries at most two people at the same time, provided the sum of their
 * weights is at most limit.
 *
 * Return the minimum number of boats to carry every given person.
 *
 * Example 1: people = [1,2], limit = 3 -> 1 (1 boat: [1,2])
 * Example 2: people = [3,2,2,1], limit = 3 -> 3 ([1,2], [2], [3])
 * Example 3: people = [3,5,3,4], limit = 5 -> 4
 *
 * Constraints:

```

```

* 1 <= people.length <= 5 * 10^4
* 1 <= people[i] <= limit <= 3 * 10^4
*
* ---
* Approach: Sort + Two pointers (greedy pairing)
*
* 1. Sort people by weight.
* 2. Use two pointers: lightest (left) and heaviest (right).
* 3. If lightest + heaviest <= limit, pair them (move both pointers).
*    Otherwise, the heaviest rides alone (move right pointer only).
* 4. Each iteration uses one boat.
*
* Greedy choice: always try to pair the heaviest remaining person with the lightest.
*
* Time: O(n log n)
* Space: O(1)
*/
public class BoatsToSavePeople {

    public int numRescueBoats(int[] people, int limit) {
        Arrays.sort(people);

        int n = people.length;
        int left = 0;
        int right = n - 1;
        int boat = 0;

        while (left <= right) {
            if (people[left] + people[right] <= limit) {
                left++;
            }
            right--;
            boat++;
        }

        return boat;
    }
}

```

Fractional Knapsack (2)

FractionalKnapsack

```
package leetcode.greedy.fractionalKnapsack;

import java.util.Arrays;

/**
 * Fractional Knapsack Problem (Classic Greedy)
 *
 * Given weights and values of n items, put these items in a knapsack of
 * capacity W to get the maximum total value. You CAN break items (take fractions).
 *
 * This is the key difference from 0/1 Knapsack (which is DP):
 * - Fractional Knapsack: greedy works (take highest value/weight ratio first)
 * - 0/1 Knapsack: greedy does NOT work (need DP)
 *
 * Example:
 * Input: values = [60, 100, 120], weights = [10, 20, 30], W = 50
 * Output: 240.0
 * Explanation:
 * Item 0: value/weight = 6.0
 * Item 1: value/weight = 5.0
 * Item 2: value/weight = 4.0
 * Take all of item 0 (10 kg, value 60), all of item 1 (20 kg, value 100),
 * and 2/3 of item 2 (20 kg, value 80). Total = 60 + 100 + 80 = 240.
 *
 * ---
 * Approach: Greedy -- sort by value/weight ratio descending
 *
 * 1. Compute value-to-weight ratio for each item.
 * 2. Sort items by ratio in descending order.
 * 3. Greedily take as much as possible of the highest-ratio item first.
 *    If the item fits entirely, take it all. Otherwise, take a fraction.
 *
 * Greedy choice property: taking the highest ratio item first always leads to
 * an optimal solution (provable by exchange argument).
 *
 * Time: O(n log n) for sorting
 * Space: O(n) for the items array
 */
public class FractionalKnapsack {

    public static double fractionalKnapsack(int[] values, int[] weights, int capacity) {
        int n = values.length;

        // Create items as [value, weight, ratio] and sort by ratio descending
        double[][] items = new double[n][3];
        for (int i = 0; i < n; i++) {
            items[i][0] = values[i];
            items[i][1] = weights[i];
            items[i][2] = (double) values[i] / weights[i];
        }

        // Sort by value/weight ratio in descending order
        Arrays.sort(items, (a, b) -> Double.compare(b[2], a[2]));

        double totalValue = 0;
        int remainingCapacity = capacity;

        for (double[] item : items) {
            double weight = item[1];
            double value = item[0];

            if (remainingCapacity >= weight) {
                // Take the whole item
                totalValue += value;
                remainingCapacity -= (int) weight;
            } else {
                // Take fraction of the item that fits
                totalValue += value * ((double) remainingCapacity / weight);
                break; // Knapsack is full
            }
        }

        return totalValue;
    }
}
```

```

    }

    public static void main(String[] args) {
        int[] values = {60, 100, 120};
        int[] weights = {10, 20, 30};
        int W = 50;
        System.out.println("Max value: " + fractionalKnapsack(values, weights, W)); // 240.0

        // Example 2: all items fit
        int[] v2 = {10, 20};
        int[] w2 = {5, 10};
        System.out.println("Max value: " + fractionalKnapsack(v2, w2, 20)); // 30.0
    }
}

```

JobSequencing

```
package leetcode.greedy.fractionalKnapsack;
```

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
```

```

/**
 * Job Sequencing Problem (Classic Greedy)
 *
 * Given a set of n jobs where each job has a deadline and a profit.
 * Each job takes 1 unit of time. Only one job can be scheduled at a time.
 * A job earns profit only if it is completed by its deadline.
 *
 * Maximize total profit.
 *
 * Example:
 * Jobs: [(1, 4, 20), (2, 1, 10), (3, 1, 40), (4, 1, 30)]
 * (id, deadline, profit)
 * Output: 2 jobs scheduled, profit = 60
 * Explanation: Schedule Job 3 (profit 40) at time 1, Job 1 (profit 20) at time 4.
 * Jobs 2 and 4 miss their deadline.
 *
 * ---
 * Approach: Greedy -- sort by profit descending, assign latest available slot
 *
 * 1. Sort jobs by profit in descending order.
 * 2. For each job, try to schedule it in the latest available time slot
 *    before its deadline.
 * 3. Use a boolean array to track which time slots are taken.
 *
 * Why latest slot? Scheduling in the latest possible slot keeps earlier
 * slots free for jobs with tighter deadlines.
 *
 * Time: O(n^2) in worst case (n jobs * searching for slot)
 *       Can be optimized to O(n log n) with Union-Find.
 * Space: O(maxDeadline)
 */

```

```

public class JobSequencing {

    public static int[] jobSequencing(int[][] jobs) {
        // Sort by profit descending
        Arrays.sort(jobs, (a, b) -> b[2] - a[2]);

        // Find the maximum deadline to determine the time slot range
        int maxDeadline = 0;
        for (int[] job : jobs) {
            maxDeadline = Math.max(maxDeadline, job[1]);
        }

        // Track which time slots are filled (1-indexed)
        boolean[] slot = new boolean[maxDeadline + 1];
        int totalProfit = 0;
        int jobCount = 0;
        List<Integer> scheduled = new ArrayList<>();

        for (int[] job : jobs) {
            int deadline = job[1];

            // Try to place this job in the latest available slot before its deadline
            for (int t = deadline; t >= 1; t--) {
                if (!slot[t]) {
                    slot[t] = true;
                    totalProfit += job[2];
                }
            }
        }
    }
}

```

```

        jobCount++;
        scheduled.add(job[0]);
        break;
    }
}

System.out.println("Scheduled job IDs: " + scheduled);
return new int[]{jobCount, totalProfit};
}

public static void main(String[] args) {
    // {id, deadline, profit}
    int[][] jobs = {
        {1, 4, 20},
        {2, 1, 10},
        {3, 1, 40},
        {4, 1, 30}
    };
    int[] result = jobSequencing(jobs);
    System.out.println("Jobs: " + result[0] + ", Profit: " + result[1]); // 2, 60

    int[][] jobs2 = {
        {1, 2, 100},
        {2, 1, 19},
        {3, 2, 27},
        {4, 1, 25},
        {5, 1, 15}
    };
    int[] result2 = jobSequencing(jobs2);
    System.out.println("Jobs: " + result2[0] + ", Profit: " + result2[1]); // 2, 127
}
}

```

Interval Problems (8)

CarPooling

```
package leetcode.greedy.intervals;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

/**
 * 1094. Car Pooling
 * https://leetcode.com/problems/car-pooling/
 *
 * There is a car with 'capacity' empty seats. The vehicle only drives east
 * (i.e., it cannot turn around and drive west).
 *
 * You are given the integer capacity and an array trips where
 * trips[i] = [numPassengers_i, from_i, to_i] indicates that the i-th trip has
 * numPassengers_i passengers and the locations to pick them up and drop them off
 * are from_i and to_i respectively.
 *
 * Return true if it is possible to pick up and drop off all passengers for all
 * the given trips, or false otherwise.
 *
 * Example 1: trips = [[2,1,5],[3,3,7]], capacity = 4 -> false
 * Example 2: trips = [[2,1,5],[3,3,7]], capacity = 5 -> true
 *
 * Constraints:
 * 1 <= trips.length <= 1000
 * 1 <= capacity <= 10^5
 * 0 <= from_i < to_i <= 1000
 *
 * ---
 * Approach: Sweep Line (event-based)
 *
 * Convert each trip into two events:
 * - Pickup at location from: +numPassengers
 * - Drop-off at location to: -numPassengers
 *
 * Sort events by location (drop-offs before pickups at same location).
 * Sweep through events, tracking current passengers. If ever > capacity, return false.
 *
 * Time: O(n log n)
 * Space: O(n)
 */
public class CarPooling {

    public static boolean carPooling(int[][] trips, int capacity) {
        List<int[]> location = new ArrayList<>();

        // Convert trips into pickup/drop-off events
        for (int[] trip : trips) {
            location.add(new int[]{trip[1], trip[0]}); // Pickup event (start location, numPassengers)
            location.add(new int[]{trip[2], -trip[0]}); // Drop-off event (end location, -numPassengers)
        }

        Comparator<int[]> comparator = (a, b) -> {
            if (a[0] == b[0]) return a[1] - b[1]; // Pickup before drop-off
            return a[0] - b[0]; // Sort by location
        };

        location.sort(comparator);
        int currentCapacity = 0;
        for (int[] trip : location) {
            currentCapacity += trip[1];

            if (currentCapacity > capacity) return false;
        }

        return true;
    }

    public static void main(String[] args) {
        int[][] trips = {{2, 1, 5}, {3, 3, 7}};
    }
}
```

```

        int capacity = 5;
        System.out.println(carPooling(trips, capacity));
    }
}

```

PartitionLabels

```

package leetcode.greedy.intervals;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 763. Partition Labels
 * https://leetcode.com/problems/partition-labels/
 *
 * You are given a string s. We want to partition the string into as many parts
 * as possible so that each letter appears in at most one part.
 *
 * Note that the partition is done so that after concatenating all the parts in
 * order, the resultant string should be s.
 *
 * Return a list of integers representing the size of these parts.
 *
 * Example 1:
 * Input: s = "ababcbacadefegdehijhklij"
 * Output: [9,7,8]
 * Explanation: "ababcbaca", "defegde", "hijhklij"
 * Each letter appears in at most one part.
 *
 * Example 2: s = "eccbbbbbdec" -> [10]
 *
 * Constraints:
 * 1 <= s.length <= 500
 * s consists of lowercase English letters.
 *
 * ---
 * Approach: Greedy -- extend partition to last occurrence
 *
 * 1. Record the last index of each character.
 * 2. Iterate through s, tracking the farthest last occurrence ("end") of any
 *    character seen so far in the current partition.
 * 3. When i == end, we've found a valid partition boundary. Record its size.
 *
 * Think of each character as an interval [first, last]. The problem becomes
 * merging overlapping intervals and reporting their sizes.
 *
 * Time: O(n)
 * Space: O(1) (26 letters)
 */
public class PartitionLabels {

    public static List<Integer> partitionLabels(String s) {
        Map<Character, Integer> lastSeenIndex = new HashMap<>();

        for (int i = 0; i < s.length(); i++) {
            lastSeenIndex.put(s.charAt(i), i);
        }

        int start = 0;
        int end = Integer.MIN_VALUE;
        List<Integer> result = new ArrayList<>();

        for (int i = 0; i < s.length(); i++) {
            if (lastSeenIndex.get(s.charAt(i)) > end) {
                end = lastSeenIndex.get(s.charAt(i));
            }

            if (end == i) {
                result.add(i - start + 1);
                start = i + 1;
            }
        }

        return result;
    }
}

```

```

    public static void main(String[] args) {
        String s = "ababcbacadeefegdehijhklj";
        System.out.println(partitionLabels(s));
    }
}

```

InsertIntervals_57

```

package leetcode.greedy.intervals.merging;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/*
 * LeetCode 57: Insert Interval
 *
 * You are given an array of non-overlapping intervals `intervals` where
 * `intervals[i] = [start_i, end_i]` represent the start and the end of the
 * i-th interval and `intervals` is sorted in ascending order by `start_i`.
 *
 * You are also given an interval `newInterval = [start, end]` that
 * represents the start and end of another interval.
 *
 * Insert `newInterval` into `intervals` such that `intervals` is still
 * sorted in ascending order by `start_i` and `intervals` still does not
 * have any overlapping intervals (merge overlapping intervals if necessary).
 *
 * Return `intervals` after the insertion.
 *
 * ---
 *
 * Example 1:
 * Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
 * Output: [[1,5],[6,9]]
 *
 * Example 2:
 * Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
 * Output: [[1,2],[3,10],[12,16]]
 * Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].
 *
 * Example 3 (Edge Case):
 * Input: intervals = [], newInterval = [5,7]
 * Output: [[5,7]]
 *
 * ---
 *
 * Constraints:
 * 1. 0 <= intervals.length <= 10^4
 * 2. intervals[i].length == 2
 * 3. 0 <= start_i <= end_i <= 10^5
 * 4. intervals is sorted by start_i in ascending order.
 * 5. newInterval.length == 2
 * 6. 0 <= start <= end <= 10^5
 */
@SuppressWarnings("all")
public class InsertIntervals_57 {

    public static int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> result = new ArrayList<>();

        for (int[] current : intervals) {
            // Case 1: Current is "before" New (or New has already been placed)
            // `newInterval == null` handles all intervals *after* Case 2 has occurred.
            if (newInterval == null || current[1] < newInterval[0]) {
                result.add(current);
                // Case 2: Current is "after" New
            } else if (current[0] > newInterval[1]) {
                // This is the first time we've seen an interval *after* newInterval.
                // Add the (potentially merged) newInterval.
                result.add(newInterval);
                // Add the current interval that comes after it.
                result.add(current);
                // Mark newInterval as placed.
                newInterval = null;
                // Case 3: Overlap
            } else {

```

```

        // Merge current into newInterval by updating newInterval's bounds.
        newInterval[0] = Math.min(newInterval[0], current[0]);
        newInterval[1] = Math.max(newInterval[1], current[1]);
    }
}

// If newInterval was never placed (i.e., it belongs at the end,
// or the list was empty), add it now.
if (newInterval != null) {
    result.add(newInterval);
}

return result.toArray(new int[result.size()][]);
}

public static void main(String[] args) {
    // NOTE: The first example here is *unsorted* and does not follow
    // the problem's constraints. The algorithm assumes a sorted input array.
    // If it were sorted as {{1, 3}, {8, 10}, {15, 18}}, the output would be:
    // [[1, 6], [8, 10], [15, 18]]
    int[][] intervals = {{1, 3}, {15, 18}, {8, 10}};
    int[] newInterval = {2, 6};
    System.out.println("Unsorted example (violates constraints): " + Arrays.deepToString(insert(intervals, newInterval)));

    // This is a valid, sorted example.
    // Input: [[1, 3], [15, 18]], newInterval = [4, 6]
    // Output: [[1, 3], [4, 6], [15, 18]]
    intervals = new int[][]{{1, 3}, {15, 18}};
    newInterval = new int[]{4, 6};
    System.out.println("Valid sorted example: " + Arrays.deepToString(insert(intervals, newInterval)));

    // Official Example 1:
    // Input: [[1, 3], [6, 9]], newInterval = [2, 5]
    // Output: [[1, 5], [6, 9]]
    intervals = new int[][]{{1, 3}, {6, 9}};
    newInterval = new int[]{2, 5};
    System.out.println("Official Example 1: " + Arrays.deepToString(insert(intervals, newInterval)));

    // Official Example 2:
    // Input: [[1, 2], [3, 5], [6, 7], [8, 10], [12, 16]], newInterval = [4, 8]
    // Output: [[1, 2], [3, 10], [12, 16]]
    intervals = new int[][]{{1, 2}, {3, 5}, {6, 7}, {8, 10}, {12, 16}};
    newInterval = new int[]{4, 8};
    System.out.println("Official Example 2: " + Arrays.deepToString(insert(intervals, newInterval)));
}
}

```

MergeIntervals_56

```
package leetcode.greedy.intervals.merging;
```

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
```

```

/*
 * LeetCode 56: Merge Intervals
 *
 * Given an array of intervals where `intervals[i] = [start_i, end_i]`,
 * merge all overlapping intervals, and return an array of the
 * non-overlapping intervals that cover all the intervals in the input.
 *
 * ---
 *
 * Example 1:
 * Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
 * Output: [[1,6],[8,10],[15,18]]
 * Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
 *
 * Example 2:
 * Input: intervals = [[1,4],[4,5]]
 * Output: [[1,5]]
 * Explanation: Intervals [1,4] and [4,5] are considered overlapping.
 *
 * ---
 *
 * Constraints:
 * 1. 1 <= intervals.length <= 10^4
 * 2. intervals[i].length == 2
 * 3. 0 <= start_i <= end_i <= 10^4
 */

```

```

*/
public class MergeIntervals_56 {

    public static int[][] merge(int[][] intervals) {
        // 1. Sort the intervals by start time.
        // Example: [[1,3],[2,6],[15,18],[8,10]] -> [[1,3],[2,6],[8,10],[15,18]]
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        List<int[]> result = new ArrayList<>();
        // 2. Initialize the result list with the first interval.
        result.add(intervals[0]); // the result is now [[1,3]]

        // 3. Iterate through all intervals
        for (int[] currentInterval : intervals) {
            int[] lastInterval = result.getLast();

            // 4. Check for Overlap
            if (currentInterval[0] <= lastInterval[1]) {
                result.removeLast();
                int min = Math.min(lastInterval[0], currentInterval[0]);
                int max = Math.max(lastInterval[1], currentInterval[1]);
                result.add(new int[]{min, max});
            } else {
                result.add(currentInterval);
            }
        }

        // 6. Return the result
        return result.toArray(new int[result.size()][]);
    }

    static void main(String[] args) {
        int[][] intervals = {{1, 3}, {2, 6}, {15, 18}, {8, 10}};
        int[][] result = merge(intervals);
        // After sorting: [[1, 3], [2, 6], [8, 10], [15, 18]]
        // After merging: [[1, 6], [8, 10], [15, 18]]
        System.out.println(Arrays.deepToString(result));
    }
}

```

MeetingRooms2_253

```
package leetcode.greedy.intervals.overlap;
```

```
import java.util.Arrays;
import java.util.PriorityQueue;
```

```

/**
 * 253. Meeting Rooms II
 * https://leetcode.com/problems/meeting-rooms-ii/
 *
 * Given an array of meeting time intervals where intervals[i] = [start_i, end_i],
 * return the minimum number of conference rooms required.
 *
 * Example 1:
 * Input: intervals = [[0,30],[5,10],[15,20]]
 * Output: 2
 * Explanation: [0,30] overlaps with [5,10] and [15,20], but [5,10] and [15,20]
 * don't overlap, so 2 rooms are needed.
 *
 * Example 2:
 * Input: intervals = [[7,10],[2,4]]
 * Output: 1
 *
 * Constraints:
 * 1 <= intervals.length <= 10^4
 * 0 <= start_i < end_i <= 10^6
 *
 * ---
 * Approach: Sort + Min-Heap (greedy room reuse)
 *
 * 1. Sort meetings by start time.
 * 2. Use a min-heap to track end times of ongoing meetings.
 * 3. For each meeting:
 *    - If the earliest-ending meeting finishes before this one starts -> reuse that room (poll).
 *    - Add this meeting's end time to the heap.
 * 4. Heap size = number of rooms needed.
 *
 * Greedy choice: always reuse the room that frees up earliest.
 *
 * Time: O(n log n)
 */

```

```

* Space: O(n)
*/
public class MeetingRooms2_253 {

    /*
    * Problem Statement
    * Given an array of meeting time intervals where each interval is represented
    * as an array of two integers [start, end], find the minimum number of conference rooms
    * required to hold all meetings.
    *
    * Example
    * Input: [[0, 30], [5, 10], [15, 20]]
    * Output: 2
    *
    * In the above example:
    * You need two rooms because [0, 30] overlaps with [5, 10] and [15, 20].
    */
    public static int minMeetingRooms(int[][] intervals) {
        if (intervals == null || intervals.length == 0) {
            return 0;
        }

        // Sort the intervals by their start times
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        // Use a min-heap to track the end time of meetings
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        // Add the first meeting's end time to the heap
        minHeap.add(intervals[0][1]);

        for (int i = 1; i < intervals.length; i++) {
            // If the earliest ending meeting ends before the next meeting starts
            if (intervals[i][0] >= minHeap.peek()) {
                // Remove the earliest ending meeting
                minHeap.poll();
            }
            // Add the current meeting's end time to the heap
            minHeap.add(intervals[i][1]);
        }

        // The size of the heap tells us the minimum rooms required
        return minHeap.size();
    }

    public static void main(String[] args) {
        int[][] intervals = {{0, 30}, {5, 10}, {15, 20}};
        System.out.println(minMeetingRooms(intervals)); // Output: 2
    }
}

```

MeetingRooms_252

```
package leetcode.greedy.intervals.overlap;
```

```
import java.util.Arrays;
```

```

/*
* LeetCode 252: Meeting Rooms
*
* Given an array of meeting time intervals `intervals` where
* `intervals[i] = [start_i, end_i]`, determine if a person could
* attend all meetings.
*
* ---
*
* Example 1:
* Input: intervals = [[0,30],[5,10],[15,20]]
* Output: false
* Explanation: [0,30] overlaps with [5,10] and [15,20].
*
* Example 2:
* Input: intervals = [[7,10],[2,4]]
* Output: true
* Explanation: [2,4] and [7,10] do not overlap.
*
* ---
*
* Constraints:

```

```

* 1. 0 <= intervals.length <= 10^4
* 2. intervals[i].length == 2
* 3. 0 <= start_i <= end_i <= 10^6
*/
public class MeetingRooms_252 {

    public static boolean canAttendMeetings(int[][] intervals) {
        // 1. Handle edge case for 0 or 1 meetings.
        if (intervals == null || intervals.length <= 1) {
            return true;
        }

        // 2. Sort the intervals by their start time (a[0]).
        // Example 1 becomes: [[0, 30], [5, 10], [15, 20]]
        // Example 2 becomes: [[2, 4], [7, 10]]
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        // 3. Iterate from the second meeting (i=1).
        for (int i = 1; i < intervals.length; i++) {
            int currentMeetingStart = intervals[i][0];
            int previousMeetingEnd = intervals[i - 1][1];

            // 4. Check for overlap.
            // If the current meeting starts *before* the previous one ends,
            // they conflict.
            if (currentMeetingStart < previousMeetingEnd) {
                // Example 1, i=1:
                // currentStart = 5, previousEnd = 30
                // (5 < 30) is true. Return false.
                return false;
            }
            // Example 2, i=1:
            // currentStart = 7, previousEnd = 4
            // (7 < 4) is false. Loop continues.
        }

        // 5. If the loop completes, no overlaps were found.
        return true;
    }

    static void main(String[] args) {
        // Example 1: [[0, 30], {5, 10}, {15, 20}]
        // Sorted: [[0, 30], [5, 10], [15, 20]]
        // Check i=1: [5, 10] vs [0, 30]. Is 5 < 30? Yes. Return false.
        int[][] intervals = {{0, 30}, {5, 10}, {15, 20}};
        System.out.println(canAttendMeetings(intervals)); // Output: false

        // Example 2: [[7, 10], [2, 4]]
        // Sorted: [[2, 4], [7, 10]]
        // Check i=1: [7, 10] vs [2, 4]. Is 7 < 4? No.
        // Loop finishes. Return true.
        intervals = new int[][]{{7, 10}, {2, 4}};
        System.out.println(canAttendMeetings(intervals)); // Output: true
    }
}

```

MinimumArrowsBurstBalloons

```
package leetcode.greedy.intervals.scheduling;
```

```
import java.util.Arrays;
```

```

/**
 * 452. Minimum Number of Arrows to Burst Balloons
 * https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/
 *
 * There are some spherical balloons taped onto a flat wall represented by the
 * XY-plane. Balloons are represented as a 2D integer array points where
 * points[i] = [x_start, x_end] denotes a balloon whose horizontal diameter
 * stretches between x_start and x_end. You do not know the exact y-coordinates.
 *
 * Arrows can be shot up vertically (in the positive y-direction) from different
 * points along the x-axis. A balloon with x_start <= arrow <= x_end bursts.
 *
 * Return the minimum number of arrows to burst all balloons.
 *
 * Example 1:
 * Input: points = [[10,16],[2,8],[1,6],[7,12]]
 * Output: 2
 * Explanation: Shoot at x=6 (bursts [2,8] and [1,6]) and x=11 (bursts [10,16] and [7,12]).
 */

```

```

* Example 2:
*   Input: points = [[1,2],[3,4],[5,6],[7,8]]
*   Output: 4 (no overlaps)
*
* Example 3:
*   Input: points = [[1,2],[2,3],[3,4],[4,5]]
*   Output: 2 (shoot at x=2 and x=4)
*
* Constraints:
*   1 <= points.length <= 10^5
*   points[i].length == 2
*   -2^31 <= x_start <= x_end <= 2^31 - 1
*
* ---
* Approach: Greedy -- sort by end, count non-overlapping groups
*
* This is equivalent to the classic "Activity Selection" / "Interval Scheduling
* Maximization" problem:
*   - Sort balloons by their END coordinate.
*   - Greedily shoot an arrow at the end of the first balloon.
*   - Skip all balloons that this arrow also bursts (overlap with arrow position).
*   - When a balloon starts AFTER the arrow, we need a new arrow.
*
* This is the complement of NonOverlappingIntervals (LC 435):
*   - LC 435: count intervals to REMOVE = total - maxNonOverlapping
*   - LC 452: count groups of overlapping intervals = arrows needed
*
* Time: O(n log n)
* Space: O(1)
*/
public class MinimumArrowsBurstBalloons {

    public static int findMinArrowShots(int[][] points) {
        // Sort by end coordinate
        // Use Integer.compare to avoid overflow with extreme int values
        Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));

        int arrows = 1;
        int arrowPos = points[0][1]; // Shoot first arrow at end of first balloon

        for (int i = 1; i < points.length; i++) {
            // If this balloon starts after the current arrow position,
            // it wasn't burst -> need a new arrow
            if (points[i][0] > arrowPos) {
                arrows++;
                arrowPos = points[i][1]; // Shoot at end of this balloon
            }
            // Otherwise, this balloon overlaps and is already burst
        }

        return arrows;
    }

    public static void main(String[] args) {
        int[][] points1 = {{10, 16}, {2, 8}, {1, 6}, {7, 12}};
        System.out.println("Example 1: " + findMinArrowShots(points1)); // 2

        int[][] points2 = {{1, 2}, {3, 4}, {5, 6}, {7, 8}};
        System.out.println("Example 2: " + findMinArrowShots(points2)); // 4

        int[][] points3 = {{1, 2}, {2, 3}, {3, 4}, {4, 5}};
        System.out.println("Example 3: " + findMinArrowShots(points3)); // 2
    }
}

```

NonOverlappingIntervals_435

```
package leetcode.greedy.intervals.scheduling;
```

```
import java.util.Arrays;
```

```

/*
 * LeetCode 435: Non-overlapping Intervals
 *
 * Given an array of intervals `intervals` where `intervals[i] = [start_i, end_i]`,
 * return the minimum number of intervals you need to remove to make the rest
 * of the intervals non-overlapping.
 *
 * ---
 */

```

```

*
* Example 1:
* Input: intervals = [[1,2],[2,3],[3,4],[1,3]]
* Output: 1
* Explanation: [1,3] can be removed and the rest of the intervals are
* non-overlapping.
*
* Example 2:
* Input: intervals = [[1,2],[1,2],[1,2]]
* Output: 2
* Explanation: You need to remove two [1,2] to make the rest of the
* intervals non-overlapping.
*
* Example 3:
* Input: intervals = [[1,2],[2,3]]
* Output: 0
* Explanation: You don't need to remove any of the intervals since they're
* already non-overlapping.
*
* ---
*
* Constraints:
* 1. 1 <= intervals.length <= 10^5
* 2. intervals[i].length == 2
* 3. -5 * 10^4 <= start_i < end_i <= 5 * 10^4
*/
public class NonOverlappingIntervals_435 {

    public static int eraseOverlapIntervals(int[][] intervals) {
        // Handle edge case
        if (intervals.length == 0) return 0;

        // 1. Sort by end time
        // Example: [[1,2], [2,3], [3,4], [1,3]]
        // Sorted: [[1,2], [2,3], [1,3], [3,4]] - Ascending by end time

        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);

        // 2. Initialize: Keep the first interval
        int end = intervals[0][1]; // end = 2 (from [1,2])
        int count = 1; // count = 1 (we kept [1,2])

        // 3. Iterate and Select
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] >= end) {
                end = intervals[i][1];
                count++;
            }
        }

        // 4. Return result
        // Total intervals = 4
        // Kept intervals (count) = 3
        // Removed = 4 - 3 = 1
        return intervals.length - count;
    }

    public static void main(String[] args) {
        int[][] intervals = {{1, 2}, {2, 3}, {3, 4}, {1, 3}};
        System.out.println(eraseOverlapIntervals(intervals)); // Output: 1
    }
}

```